🏗️ Step-by-Step Backend Build Guide — Degree Certificate Verification System

> This guide walks you through building the **FastAPI backend** that connects your existing React frontend to the algorithm already present in your Jupyter notebook (`degree_verification_v2_TESSERACT.ipynb`).

📍 Where You Stand Right Now

Layer	Status
Frontend (React + Vite + TypeScript)	✅ Built — 6 pages, type definitions, components, routing. Currently uses **empty mock data objects** `{}` and has most render logic **commented out**. No `fetch` / `axios` calls exist yet.
Algorithm (Jupyter Notebook)	✅ Written — PDF parsing via `pdfplumber`, Kruti Dev → Unicode transcoding, Hindi normalization, Tesseract OCR fallback, Excel roster lookup, multi-tier field matching, verification verdict logic.
Backend API	❌ Does not exist yet
Database	❌ Excel file only — not in a queryable DB
OpenCV alignment / heatmap	❌ Referenced in design doc and UI placeholders but not yet implemented

Overview of the Data Flow You Need to Build

STEP 1: Set Up the Backend Project Structure

Create a `backend/` folder at the same level as your `frontend/` folder.

Folder structure to create:

```
degree_verify_not_github/
├── frontend/          ← (already exists)
├── backend/
│   ├── app/
│   │   ├── __init__.py
│   │   ├── main.py              ← FastAPI app entry point
│   │   ├── config.py            ← settings (paths, thresholds)
│   │   ├── models/
│   │   │   ├── __init__.py
│   │   │   ├── schemas.py       ← Pydantic models matching frontend types
│   │   │   └── database.py      ← SQLite ORM models
│   │   ├── routers/
│   │   │   ├── __init__.py
│   │   │   ├── upload.py        ← POST /api/certificates/upload
│   │   │   ├── verification.py  ← GET  /api/verification/{id}/status
│   │   │   ├── certificates.py  ← GET  /api/certificates/{id}
│   │   │   ├── dashboard.py     ← GET  /api/dashboard/stats & /recent
│   │   │   └── review.py        ← POST /api/review/{id}/action & GET /queue
│   │   ├── services/
│   │   │   ├── __init__.py
│   │   │   ├── pdf_parser.py    ← extract from notebook: pdf_lines, parse_page
│   │   │   ├── kruti_dev.py     ← extract from notebook: kd2uni, KD table
│   │   │   ├── hindi_utils.py   ← extract from notebook: hn, bag, soft, cmp_hi, sim
│   │   │   ├── ocr_service.py   ← extract from notebook: ocr_lines (Tesseract)
│   │   │   ├── roster.py        ← extract from notebook: load_db, load_vocab
│   │   │   ├── verifier.py      ← extract from notebook: verify_page, overall
│   │   │   └── scoring.py       ← confidence aggregation & verdict routing
│   │   └── db/
│   │       ├── init_db.py       ← script to import Excel → SQLite
│   │       └── degree_verify.db ← SQLite database (auto-generated)
│   ├── uploads/                 ← uploaded certificates stored here
│   ├── requirements.txt
│   ├── .env                     ← environment variables
│   └── README.md
```

What to do:

1. Create the `backend/` directory and all subfolders listed above
2. Create empty `__init__.py` files in every Python package folder

STEP 2: Create `requirements.txt`

Create `backend/requirements.txt` with these dependencies:

```
fastapi>=0.115.0
uvicorn[standard]>=0.30.0
python-multipart>=0.0.9
pdfplumber>=0.11.0
pytesseract>=0.3.10
Pillow>=10.0.0
openpyxl>=3.1.0
pandas>=2.0.0
rapidfuzz>=3.0.0
python-dotenv>=1.0.0
aiosqlite>=0.20.0
sqlalchemy>=2.0.0
```

What to do:

1. Open a terminal in `backend/`
2. Run: `pip install -r requirements.txt`
3. Make sure Tesseract OCR is installed on your system:
    - Download from: https://github.com/UB-Mannheim/tesseract/wiki
    - During install, check **Hindi** (`hin`) language data
    - Add Tesseract to your system PATH, or note its install path (e.g. `C:\Program Files\Tesseract-OCR\tesseract.exe`)

STEP 3: Extract the Algorithm from the Notebook into Python Modules

This is the **most critical step**. You need to take code cells from `degree_verification_v2_TESSERACT.ipynb` and organize them into the service modules.

3a. `backend/app/services/kruti_dev.py`

- Copy the `KD` lookup table (Kruti Dev → Unicode mapping dictionary) from the notebook
- Copy the `kd2uni(raw)` function
- Copy the `_fix_spaces()` helper

- Copy the `HI_FONT` regex pattern: `re.compile(r'Ajay|Kruti|Devlys|Chanakya|Kokila|Mangal', re.I)`

3b. `backend/app/services/hindi_utils.py`

- Copy `hn(s)` — Unicode NFC normalization, strips ZWJ/ZWNJ
- Copy `bag(s)` — order-insensitive glyph-bag comparison
- Copy `soft(s)` — NFD normalization ignoring anusvara/chandrabindu/nukta differences
- Copy `sim(e, f)` — similarity percentage calculation
- Copy `cmp_hi(expected, found, ...)` — multi-tier Hindi matching function (exact → bag → soft → FAIL)

3c. `backend/app/services/pdf_parser.py`

- Copy `pdf_lines(page)` — character-level PDF parsing with line grouping
- Copy `parse_page(lines)` — template slot parser that finds anchors and extracts name/degree/roll fields
- Copy the `CONFIG` dictionary with thresholds (`COLUMN_GAP` , vertical threshold, etc.)
- Import `kd2uni` from `kruti_dev.py` and `HI_FONT` regex

3d. `backend/app/services/ocr_service.py`

- Copy `ocr_lines(page, boxes)` — Tesseract OCR fallback function
- Configure the Tesseract path from environment variable
- This function crops line bounding boxes with 6px padding at 300 DPI and runs `pytesseract.image_to_string(img, lang='hin', config='--psm 7')`

3e. `backend/app/services/roster.py`

- Copy `load_db(excel_path)` — reads the Excel workbook and builds the student lookup dict
- Copy `load_vocab(excel_path)` — reads the "Hindi Degree Name" sheet for vocabulary
- Modify to also support loading from SQLite (after Step 4)

3f. `backend/app/services/verifier.py`

- Copy `verify_page(page, db, vocab)` — the main verification function
- Copy `overall(results)` — aggregation logic (FAIL if any field fails, REVIEW if any requires review, else PASS)
- Copy `extract_certificate(page)` — standalone extraction without DB

- This module should import from all the other service modules

3g. `backend/app/services/scoring.py`

- Extract the confidence scoring and routing logic
- Map the notebook's PASS/REVIEW/FAIL to the frontend's `VerificationStatusType` (`verified` / `requires_review` / `rejected`)
- Map field-level statuses to `FieldMatchStatus` (`match` / `review` / `mismatch`)
- Calculate `overallConfidence` as a percentage from field similarity scores

> **IMPORTANT**
>
> When extracting code from the notebook, you MUST:
>
> 1. Remove all Jupyter-specific `display()` / `print()` debug statements
> 2. Replace any hardcoded file paths with config/environment variables
> 3. Add proper function signatures with type hints
> 4. Convert global variables into function parameters or class attributes
> 5. Handle errors with try/except blocks instead of letting them crash

STEP 4: Set Up the SQLite Database

Create `backend/app/db/init_db.py` — a one-time script that reads your Excel roster and inserts it into SQLite.

What this script should do:

1. Read `Degree Verification_Convocation 2026 (1).xlsx` using `openpyxl` or `pandas`
2. Parse all sheets: `EG_Phase 0`, `BTech`, `BSc`, `MA`, `MSc`, `MTech`, `PGDIIT`, `PhD`, `e-Masters`
3. Create a `students` table:

```sql
CREATE TABLE students (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    roll_number TEXT UNIQUE NOT NULL,
    english_name TEXT NOT NULL,
    hindi_name TEXT,
    programme TEXT,
    degree TEXT,
    branch TEXT,
    honours TEXT,
    minors TEXT
);
```

4. Create a `hindi_vocab` table:

```sql
CREATE TABLE hindi_vocab (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    category TEXT NOT NULL,       -- 'prog', 'disc', 'minor', 'hon'
    english_term TEXT NOT NULL,
    hindi_term TEXT NOT NULL
);
```

5. Create a `certificates` table to track uploaded & verified certificates:

```sql
CREATE TABLE certificates (
    id TEXT PRIMARY KEY,
    file_name TEXT,
    file_path TEXT,
    file_size INTEGER,
    file_type TEXT,
    upload_time TEXT,
    roll_number TEXT,
    overall_confidence REAL,
    status TEXT DEFAULT 'pending',
    review_reason TEXT,
    verification_result TEXT,      -- JSON blob of full results
    reviewed_by TEXT,
    reviewed_at TEXT,
    review_notes TEXT,
    FOREIGN KEY (roll_number) REFERENCES students(roll_number)
);
```

6. Insert all students from Excel into the `students` table
7. Insert vocabulary from the `Hindi Degree Name` sheet into `hindi_vocab`

What to do:

1. Write the script
2. Run it once: `python -m app.db.init_db`
3. Verify the DB was created: `backend/app/db/degree_verify.db`

STEP 5: Define Pydantic Schemas

Create `backend/app/models/schemas.py` with Pydantic models that **exactly match** your frontend TypeScript types.

Models to define (mirror your `frontend/src/types/`):

Frontend Type	Pydantic Model
`CertificateField`	`CertificateFieldSchema` — `name, expectedValue, extractedValue, matchStatus, confidence, matchType, reason`
`LayoutRegion`	`LayoutRegionSchema` — `name, status, expectedPosition, actualPosition, deviation`
`StudentRecord`	`StudentRecordSchema` — `rollNumber, englishName, hindiName, degree, branch, dateOfIssue, cgpa, convocationNumber`
`Certificate`	`CertificateSchema` — includes all above plus `id, fileName, overallConfidence, status, reviewReason`
`VerificationStep`	`VerificationStepSchema` — `id, name, description, status, detail`
`VerificationPipeline`	`VerificationPipelineSchema` — `certificateId, steps, currentStep, isComplete`
`ReviewItem`	`ReviewItemSchema`
`ReviewAction`	`ReviewActionSchema` — `decision, notes, reviewedBy, reviewedAt`
`DashboardStats`	`DashboardStatsSchema` — `totalCertificates, verified, requiresReview, pending`
`RecentVerification`	`RecentVerificationSchema`

TIP

> Use `model_config = ConfigDict(populate_by_name=True, alias_generator=to_camel)` or manual `Field(alias="...")` to ensure JSON keys use camelCase matching the frontend.

STEP 6: Build the FastAPI Routers (API Endpoints)

6a. `main.py` — App Entry Point

```
python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.routers import upload, verification, certificates, dashboard, review

app = FastAPI(title="Degree Certificate Verification API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],  # your Vite dev server
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(dashboard.router,     prefix="/api/dashboard")
app.include_router(upload.router,        prefix="/api/certificates")
app.include_router(verification.router,  prefix="/api/verification")
app.include_router(certificates.router,  prefix="/api/certificates")
app.include_router(review.router,        prefix="/api/review")
```

6b. Endpoints to Build

#	File	Endpoint	Method	What It Does
1	`dashboard.py`	`/api/dashboard/stats`	`GET`	Query `certificates` table, count by status, return `DashboardStats`
2	`dashboard.py`	`/api/dashboard/recent`	`GET`	Return last 10 certificates with student info as `RecentVerification[]`
3	`upload.py`	`/api/certificates/upload`	`POST`	Accept `multipart/form-data` file, save to `uploads/`, generate UUID, create DB record with status= `pending`, **kick off verification** (sync or background task), return `{ certificateId, status, message }`
4	`verification.py`	`/api/verification/{id}/status`	`GET`	Return the `VerificationPipeline` with steps and their current statuses
5	`certificates.py`	`/api/certificates/{id}`	`GET`	Return full `Certificate` object with all field results, student record, confidence
6	`review.py`	`/api/review/queue`	`GET`	Return all certificates with `status='requires_review'` as `ReviewItem[]`
7	`review.py`	`/api/review/{id}/action`	`POST`	Accept `ReviewAction`, update certificate status in DB, return updated status

6c. The Core Upload → Verify Flow (Endpoint #3 in detail)

When a file is uploaded to `/api/certificates/upload`, the backend should:

1. **Save the file** to `backend/uploads/{uuid}_{original_filename}`
2. **Create a DB record** in `certificates` table with status = `pending`
3. **Run the verification pipeline** (this is where your notebook algorithm comes in):
    - Open the PDF with `pdfplumber`
    - For each page, call `pdf_lines(page)` to extract text
    - Call `parse_page(lines)` to identify fields
    - Extract Roll Number via regex
    - Look up student in SQLite `students` table
    - Run `verify_page()` comparing extracted fields vs DB record
    - Run Tesseract OCR fallback if Hindi text-layer comparison fails

- Compute per-field confidence scores
- Call `overall()` to determine PASS/REVIEW/FAIL

4. **Map results** to frontend types:
   - PASS → `status: 'verified'`
   - REVIEW → `status: 'requires_review'`
   - FAIL → `status: 'rejected'`
   - Build `CertificateField[]` array from per-field results
   - Calculate `overallConfidence` percentage
5. **Store results** in DB as JSON
6. **Return** the certificate ID and status

> NOTE
>
> For the first version, run verification **synchronously** in the upload endpoint. Later you can switch to background tasks using `fastapi.BackgroundTasks` or Celery for async processing with real-time pipeline status updates.

STEP 7: Build the Verification Pipeline Steps

Create a list of verification steps that the frontend's `Verification.tsx` page will display:

```
python
PIPELINE_STEPS = [
    {"id": 1, "name": "PDF Parsing",            "description": "Extracting text layers and
character data from the PDF"},
    {"id": 2, "name": "Font Detection",          "description": "Identifying Hindi (Kruti
Dev/Ajay) and English font streams"},
    {"id": 3, "name": "Text Extraction",         "description": "Converting legacy font
encodings to Unicode (kd2uni)"},
    {"id": 4, "name": "Roll Number Detection",   "description": "Extracting roll number via
pattern matching"},
    {"id": 5, "name": "Database Lookup",          "description": "Retrieving student record
from roster database"},
    {"id": 6, "name": "English Field Matching",   "description": "Comparing English name,
degree, branch against records"},
    {"id": 7, "name": "Hindi Field Matching",     "description": "Multi-tier Hindi
comparison (exact → bag → soft → OCR)"},
    {"id": 8, "name": "Tesseract OCR (if needed)","description": "Running OCR fallback for
ambiguous Hindi text"},
    {"id": 9, "name": "Scoring & Verdict",        "description": "Aggregating field
confidences and determining final status"},
]
```

As each step completes during verification, update its status to `completed`, `warning`, or `error`.

STEP 8: Create the `.env` Configuration File

Create `backend/.env`:

```
env
# Database
DATABASE_URL=sqlite:///./app/db/degree_verify.db

# Excel roster path (for initial import)
EXCEL_ROSTER_PATH=../Degree Verification_Convocation 2026 (1).xlsx

# Tesseract OCR
TESSERACT_CMD=C:\Program Files\Tesseract-OCR\tesseract.exe

# Upload directory
UPLOAD_DIR=./uploads

# Server
HOST=0.0.0.0
PORT=8000

# Frontend URL (for CORS)
FRONTEND_URL=http://localhost:5173
```

STEP 9: Wire Up the Frontend to the Backend

Once your backend is running, you'll need to modify the frontend to make actual API calls instead of using mock data.

9a. Install an HTTP client in the frontend

```
bash
cd frontend
npm install axios
```

9b. Create an API service layer

Create `frontend/src/services/api.ts`:

- Base URL: `http://localhost:8000/api`
- Functions: `uploadCertificate(file)`, `getDashboardStats()`, `getRecentVerifications()`, `getVerificationStatus(id)`, `getCertificate(id)`, `submitReview(id, action)`, `getReviewQueue()`

9c. Update each page to call the API

- **Dashboard.tsx**: Replace `mockDashboardStats` and `mockRecentVerifications` with API calls
- **Upload.tsx**: `handleStartVerification()` should POST the file to `/api/certificates/upload`, then navigate to `/verification?id={certificateId}`
- **Verification.tsx**: Poll `/api/verification/{id}/status` until `isComplete` is true, then navigate to `/results`
- **Results.tsx / Comparison.tsx / Review.tsx**: Fetch certificate data from `/api/certificates/{id}`
- **Review.tsx**: `handleDecision()` should POST to `/api/review/{id}/action`

9d. Uncomment the commented-out render logic

Most of the JSX in your pages is commented out because the mock data objects are empty. Once real data flows from the backend:

1. Uncomment the `.map()` calls and conditional renders in `Verification.tsx`, `Results.tsx`, `Comparison.tsx`, and `Review.tsx`
2. Remove the empty string `""` assignments (e.g., `const flaggedFields = ""` → actual `.filter()` call)

STEP 10: Run Everything

Terminal 1 — Backend:

```
bash
cd backend
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Terminal 2 — Frontend:

```
bash
cd frontend
npm run dev
```

Test the full flow:

1. Open `http://localhost:5173`

2. Go to **Upload** page
3. Upload one of the sample PDFs (e.g., `UG-Phase-1-22110075 (4).pdf`)
4. Watch the **Verification Pipeline** page update step by step
5. View **Results** with field-by-field comparison
6. If flagged, go to **Review** and approve/reject

⚡ Quick Reference: Priority Order

If you want to get something working end-to-end as fast as possible, do these in order:

Priority	Step	Effort
🔴 1	Steps 1-2: Create folder structure & install dependencies	15 min
🔴 2	Step 3: Extract notebook algorithm into Python modules	2-3 hours
🔴 3	Step 4: Set up SQLite DB from Excel	1 hour
🟡 4	Step 5: Define Pydantic schemas	45 min
🟡 5	Step 6: Build the upload endpoint with verification	2-3 hours
🟡 6	Step 6: Build remaining endpoints (dashboard, certificates, review)	1-2 hours
🟢 7	Step 9: Wire frontend to backend API	1-2 hours
🟢 8	Step 9d: Uncomment frontend render logic	30 min

Estimated total: ~10-14 hours of work

🚫 What's NOT Covered Yet (Future Work)

These are mentioned in your design doc / UI but don't have algorithm code yet:

1. **OpenCV Image Alignment** — Aligning scanned certificates to a reference template coordinate space
2. **Layout Verification** — Checking field positions against expected bounding boxes (`LayoutRegion` type exists in frontend)
3. **Pixel-level Difference Heatmap** — Visual diff between uploaded vs reference (the `Comparison.tsx` has a placeholder for this)

4. **Batch Processing** — `ilovepdf_merged.pdf` suggests multi-page/batch upload support
5. **Authentication** — No user auth system exists yet
6. **Background Task Queue** — For async verification instead of synchronous processing

These can be added incrementally after the core pipeline works.